
Fake News Detector

— By: Alyan Qureshi, Jaisumer
Sandhu, Jaival Patel —

Problem

- Many individuals struggle to adapt to the rapidly changing digital world, Due to the big amount of online content it is challenging to distinguish the credible sources and the fake ones
- This uncertainty can lead to confusion, frustration, and mistrust, highlighting the need for tools that help users navigate and verify the truth in the digital world

Dataset

- We choose the fake news detection dataset from kaggle:
<https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets>
- The dataset covers over 25,000 political and world news articles split evenly between real news from Reuters and fake news from unreliable sources flagged by Politifact and Wikipedia collected primarily from 2016 to 2017
- This dataset included two csv files one for real news and the other for fake news

News	Size (Number of articles)	Subjects	
Real-News	21417	Type	Articles size
		World-News	10145
		Politics-News	11272
Fake-News	23481	Type	Articles size
		Government-News	1570
		Middle-east	778
		US News	783
		left-news	4459
		politics	6841
		News	9050

ML Models

MultinomialNB Model

- Designed for discrete features like word counts or term frequencies, making it ideal for natural language processing tasks.
- Lightweight and quick to train, even on large datasets with thousands of articles.
- Performs surprisingly well with simple preprocessing and serves as a reliable benchmark for comparison with more complex models.

```
fake_df['label'] = 1
true_df['label'] = 0

# concatenating the data
df = pd.concat([fake_df, true_df], ignore_index=True)

# shuffling the data
df = df.sample(frac=1, random_state=0).reset_index(drop=True)

# combining the title and text columns to make it easier for TF-IDF vectorizer
df['combined_title_and_text'] = df['title'] + " " + df['text']

# picking the combined column to use as the feature matrix
X = df['combined_title_and_text']
y = df['label']

# doing a 75% train split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

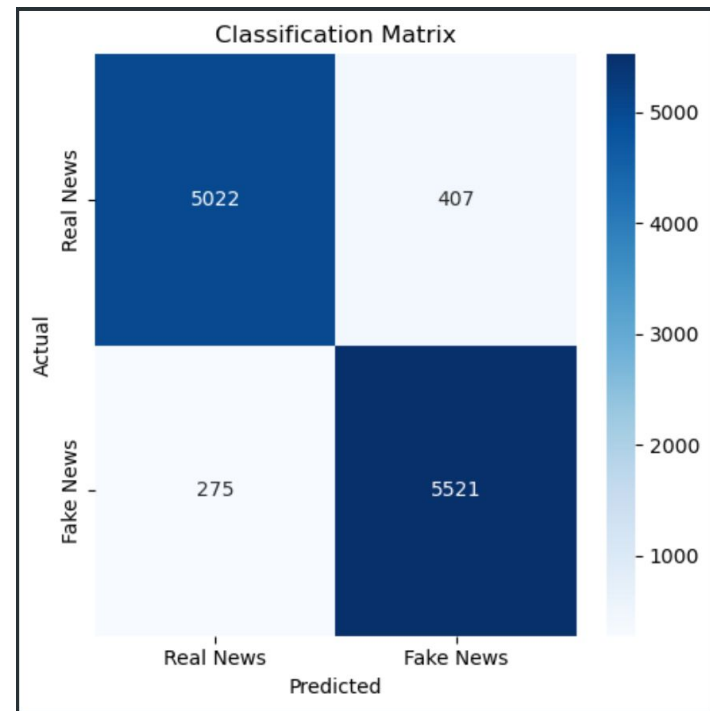
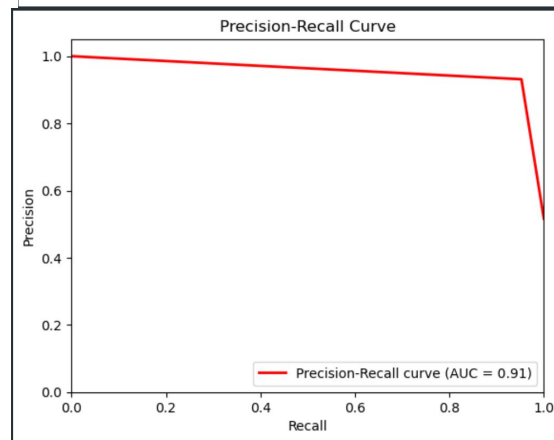
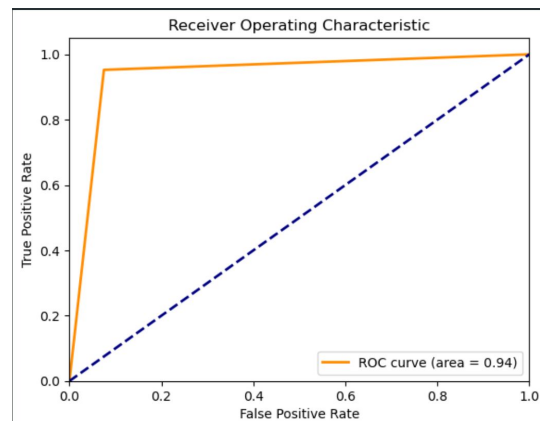
# using tfidf vectorizer as count can be misleading
# tfidf realizes common words and lowers their importance
vect = TfidfVectorizer()
X_train_vectorized = vect.fit_transform(X_train)
X_test_vectorized = vect.transform(X_test)

# instantiating multinomialNB model as its good at text classification
model = MultinomialNB()
model.fit(X_train_vectorized, y_train)

# predicting
y_pred = model.predict(X_test_vectorized)
```

MultinomialNB Model: Results

Accuracy: 0.94
Precision: 0.93
Recall: 0.95
F1-Score: 0.94
ROC-AUC: 0.94
Precision-Recall AUC: 0.91



RandomForest Model

- Uses multiple decision trees to reduce variance and handle noisy or inconsistent data effectively.
- Offers high accuracy and reliability in detecting patterns between real and fake news articles.
- Provides a ranked list of important features, helping to understand which words or attributes influence predictions most.

```
# adding labels to each df, 1 = fake news, 0 = true news
fake_df['label'] = 1
true_df['label'] = 0

# concatenating the data
df = pd.concat([fake_df, true_df], ignore_index=True)

# combining the title and text columns to make it easier for TF-IDF vectorizer
df['combined_title_and_text'] = df['title'] + " " + df['text']

# picking the combined column to use as the feature matrix
X = df['combined_title_and_text']
y = df['label']

# doing a 75% train split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

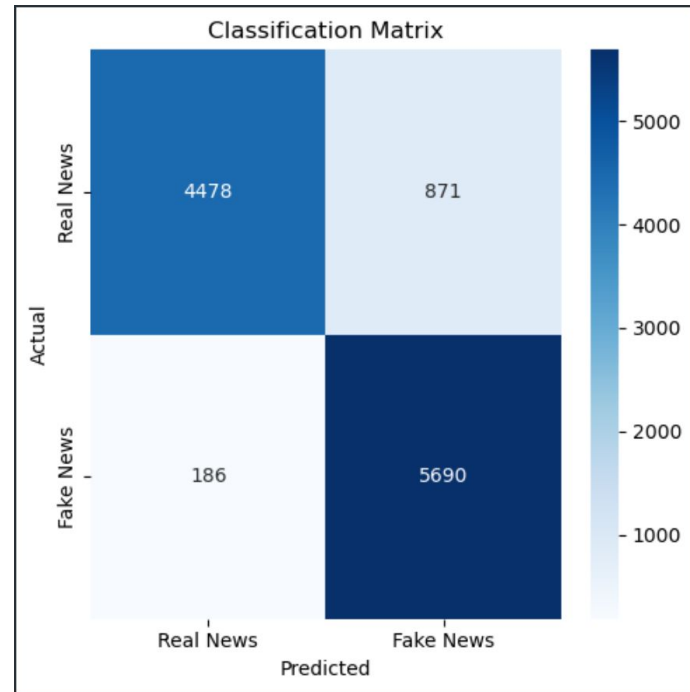
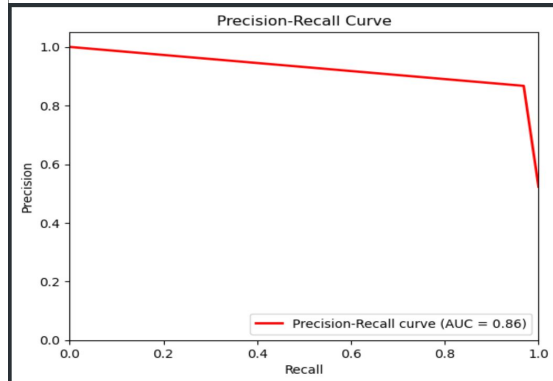
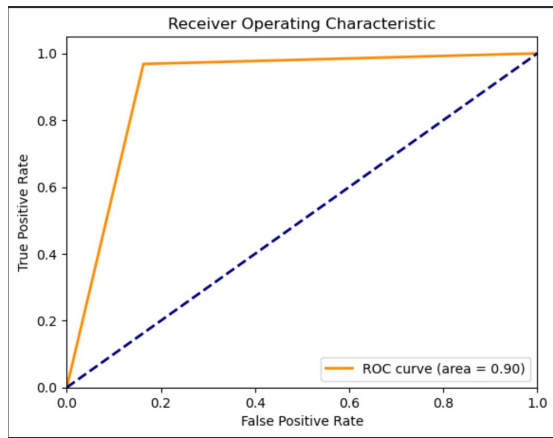
# using tfidf vectorizer as count can be misleading
# tfidf realizes common words and lowers their importance
vectorizer = TfidfVectorizer(stop_words='english', max_df=0.7)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# instantiating RandomForest model as its good at text classification
model = RandomForestClassifier(n_estimators=75, random_state=42, max_depth = 5)
model.fit(X_train_vec, y_train)

# predicting
y_pred = model.predict(X_test_vec)
```

RandomForest Model: Results

Accuracy: 0.91
Precision: 0.87
Recall: 0.97
F1-Score: 0.92
ROC-AUC: 0.90
Precision-Recall AUC: 0.86



SVM Model

- Excels in high-dimensional spaces like text data, where features are sparse and numerous
- Separates fake and real news well by maximizing the decision boundary between classes
- Can adapt to non-linear patterns in the data using different kernel functions for improved performance

```
# Adding labels to fake and true news data, where fake = 1 and true = 0
fake_df['label'] = 1
true_df['label'] = 0

# Concatenating the fake and true news data and shuffling it
df = pd.concat([fake_df, true_df], ignore_index=True)
df = df.sample(frac=1, random_state=0).reset_index(drop=True)

# Combining the title and text columns to create a single input for the vectorizer
df['combined_title_and_text'] = df['title'] + " " + df['text']
X = df['combined_title_and_text']
y = df['label']

# Splitting the data into a 75% train and 25% test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

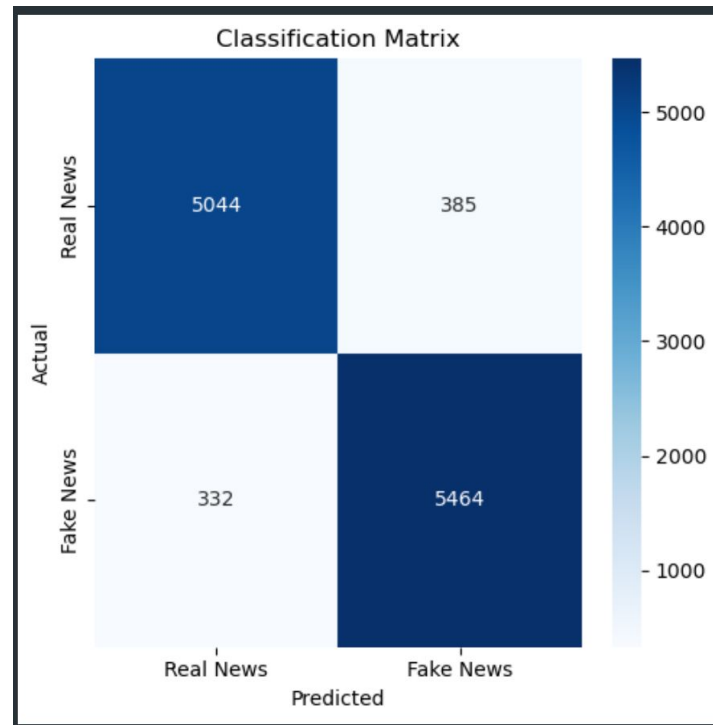
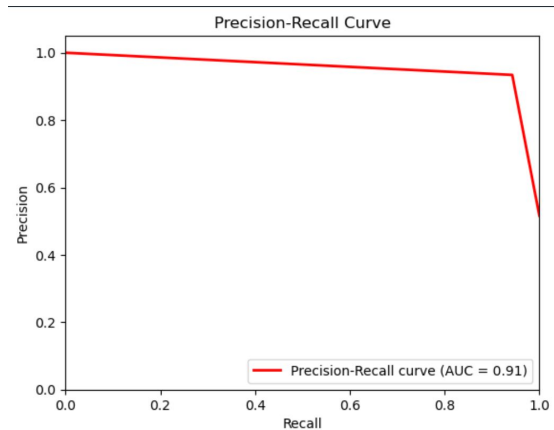
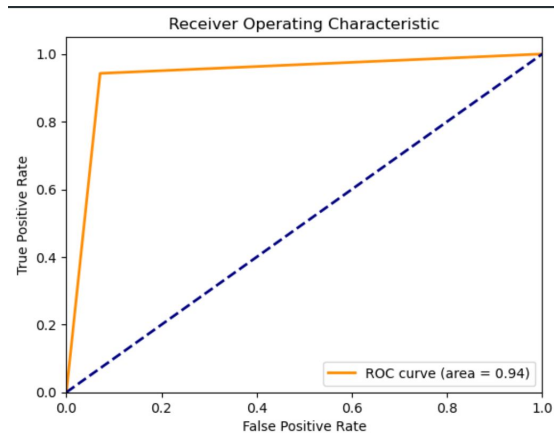
# Creating a pipeline with a TF-IDF transformer and an SVM (LinearSVC) classifier
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', max_df=0.6, max_features=1000)),
    ('svm', LinearSVC(random_state=0, C=0.001))
])

# Fitting the pipeline to the training data
pipeline.fit(X_train, y_train)

# Predicting labels for the test set
y_pred = pipeline.predict(X_test)
```

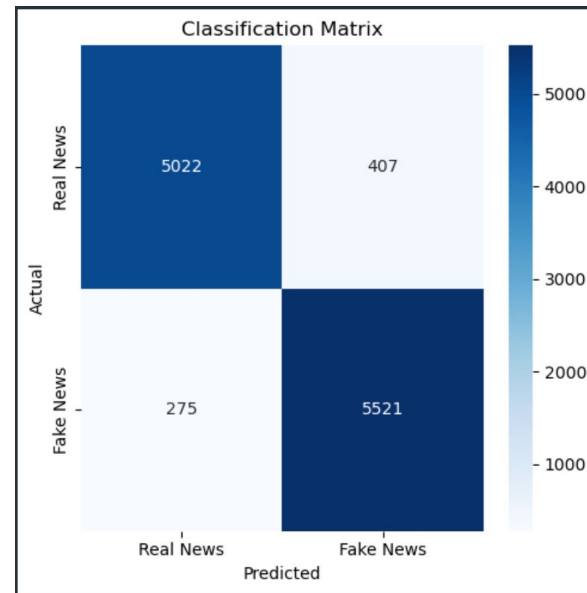
SVM Model: Results

Accuracy: 0.936125
Precision: 0.934177
Recall: 0.942719
F1-Score: 0.938429
ROC-AUC: 0.935902
Precision-Recall AUC: 0.910243



Interpretation and Impact of Results

- As seen by the multinomial confusion matrix, we prioritized making the False Negative as small as possible.
- This would mean lowering the amount of Fake news that gets labelled as True
- And the default results of each model was that the number in the bottom left (FN) was smallest as possible



Conclusion

- SVM is best for accuracy on high-dimensional, sparse text data like news articles, making it ideal for fake news detection
- MultinomialNB is fastest and works well as a simple, efficient baseline model for word-frequency-based features
- Random Forest offers interpretability and robustness to noise but may not perform as well with sparse, high-dimensional text
- In comparison for our dataset, Multinomial provided the most accurate result compared to the other models and then after that was the SVM model followed by RandomForest

THE END